



Experiment: EDSR simple gradient loss



Model

- EDSR [1] (src/models/edsr/EDSR)
 - 1 input channel
 - scale factor 10
 - the same architecture as Košťál et al. [2].



Data

- ReKIS daily mean temperature
 - training set: 1961 - 1992
 - validation set: 1993 - 2002
 - test set: 2003 - 2012



Loss function

Standard approach (HR dependent)

- Standard MAE loss ($\mathcal{L}_{MAE}$) between model output and target.
- Simple gradient loss (src/loss/loss/SimpleGradientLoss).
- Simple derivative with respect to x was calculated by convolution with kernel $\begin{bmatrix} 1 & -1 \end{bmatrix}$. Similarly, derivative with respect to y with kernel $\begin{bmatrix} 1 \\ -1 \end{bmatrix}$.
- Then the derivatives of I^{HR} and I^{SR} are:

$$\begin{aligned} D_x^{\text{HR}} &= I^{\text{HR}} \star \begin{bmatrix} 1 & -1 \end{bmatrix} & D_x^{\text{SR}} &= I^{\text{SR}} \star \begin{bmatrix} 1 & -1 \end{bmatrix} \\ D_y^{\text{HR}} &= I^{\text{HR}} \star \begin{bmatrix} 1 \\ -1 \end{bmatrix} & D_y^{\text{SR}} &= I^{\text{SR}} \star \begin{bmatrix} 1 \\ -1 \end{bmatrix} \end{aligned}$$

- Then the simple gradient loss function will be:

$$\mathcal{L}_D = MAE(D_x^{\text{HR}}, D_x^{\text{SR}}) + MAE(D_y^{\text{HR}}, D_y^{\text{SR}})$$

TODO: try MSE

- Therefore, the final loss function is:

$$\mathcal{L} = (1 - \alpha) \cdot \mathcal{L}_{MAE} + \alpha \cdot \mathcal{L}_D$$

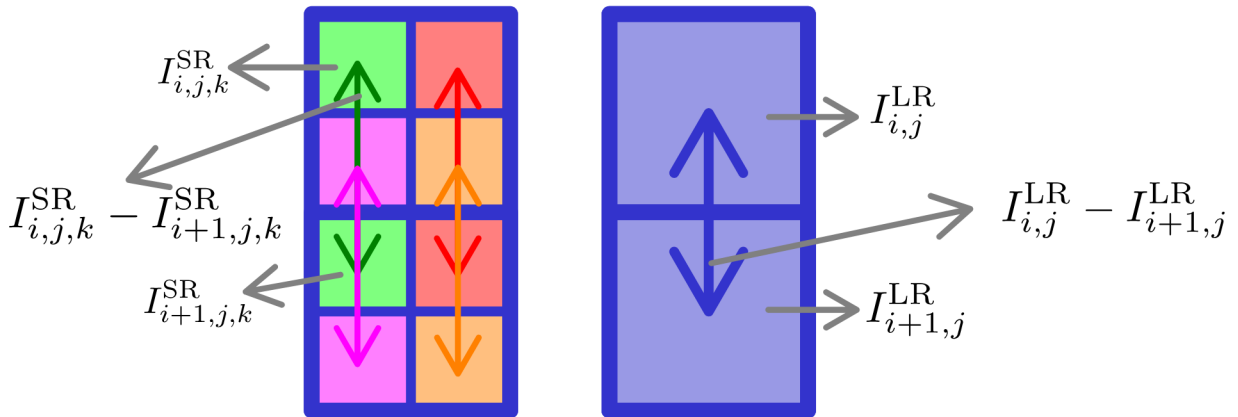
- where α controls the attention we give the gradient loss.
- for now, we used $\alpha = 0.01$ and MAE for gradient loss.

TODO: Try MSE and other α s.

Soft constraint approach (LR dependent)

- Standard MAE loss ($\mathcal{L}_{MAE}$) between model output and target.
- Soft simple gradient loss (src/loss/loss/SoftSimpleGradientLoss).
- This approach is slightly different from above. Here we have images of different sizes (LR and SR), and the gradient loss is not so straightforward as above.
- The difference from above is that instead of using raw SR derivatives, we will use mean SR differences between groups of downscaled pixels, which should be the same as LR derivatives.
- Formally, assume we have pixels indexed as $I_{i,j}^{LR}$ for LR pixel in i -th pixel in j -th row and SR pixels downscaled from $I_{i,j}^{LR}$ indexed as $\{I_{i,j,k}^{SR} \mid k \in \{1, 2, \dots, \text{scale_factor}^2\}\}$.
- Then the assumption is that the mean difference $I_{i,j,k}^{SR}$ between $I_{i+1,j,k}^{SR}$ by corresponding k is equal to the difference between corresponding LR pixels:

$$\frac{1}{\text{scale_factor}^2} \sum_{k=1}^{\text{scale_factor}^2} I_{i,j,k}^{SR} - I_{i+1,j,k}^{SR} = I_{i,j}^{LR} - I_{i+1,j}^{LR}$$



- The above equation was implemented as:

$$I_U^{SR} = \text{PixelUnshuffle}(I^{SR}, \text{scale_factor})$$

- Simple derivative with respect to x was calculated by convolution with kernel $\begin{bmatrix} 1 & -1 \end{bmatrix}$. Similarly, derivative with respect to y with kernel $\begin{bmatrix} 1 \\ -1 \end{bmatrix}$.
- Then the derivatives of I^{HR} and I_U^{SR} are:

$$\begin{aligned} D_x^{LR} &= I^{LR} \star \begin{bmatrix} 1 & -1 \end{bmatrix} & D_{U_x}^{SR} &= I_U^{SR} \star \begin{bmatrix} 1 & -1 \end{bmatrix} \\ D_y^{LR} &= I^{LR} \star \begin{bmatrix} 1 \\ -1 \end{bmatrix} & D_{U_y}^{SR} &= I_U^{SR} \star \begin{bmatrix} 1 \\ -1 \end{bmatrix} \end{aligned}$$

- Now, we calculate the mean differences

$$\bar{D}_x^{\text{SR}} = \text{Mean}(D_{U_x}^{\text{SR}}) \quad \bar{D}_y^{\text{SR}} = \text{Mean}(D_{U_y}^{\text{SR}})$$

- Then the soft simple gradient loss function will be:

$$\mathcal{L}_D = \text{MAE}(D_x^{\text{LR}}, \bar{D}_x^{\text{SR}}) + \text{MAE}(D_y^{\text{LR}}, \bar{D}_y^{\text{SR}})$$

TODO: try MSE

- Therefore, the final loss function is:

$$\mathcal{L} = (1 - \alpha) \cdot \mathcal{L}_{\text{MAE}} + \alpha \cdot \mathcal{L}_D$$

- where α controls the attention we give the gradient loss.
- for now, we used $\alpha = 0.01$ and MAE for gradient loss.

TODO: Try MSE and other α s.

- We also provide a table of dimensions, to make them more clear.

Variable	Dimension
I^{LR}	$(B, 1, H, W)$
I^{SR}	$(B, 1, H \cdot \text{scale_factor}, W \cdot \text{scale_factor})$
I_U^{SR}	$(B, \text{scale_factor}, H, W)$
$D_{U_x}^{\text{SR}}$	$(B, \text{scale_factor}, H, W - 1)$
$D_{U_y}^{\text{SR}}$	$(B, \text{scale_factor}, H - 1, W)$
$\bar{D}_x^{\text{SR}}$	$(B, 1, H, W - 1)$
$\bar{D}_y^{\text{SR}}$	$(B, 1, H - 1, W)$
D_x^{LR}	$(B, 1, H, W - 1)$
D_y^{LR}	$(B, 1, H - 1, W)$

Config

- Early stopping based on validation MAE, resp. RMSE, with patience 10, resp. 15.

Optimizer

- Adam with learning rates 1e-3, 1e-4, 1e-5.

Training

- RCI
 - partition: gpu
 - 1 node
 - 1 task per node
 - 1 GPU
 - 8 CPU cores
 - 8 dataloader workers
 - 32 GB RAM
- training logs are saved on RCI in ~/SISR-downscaling/logs/lightning_logs as **version_{3, 13, 23, 27, 28, 29, 30, 31, 32}**.

Result

#experiment		lowest validation MAE	lowest validation RMSE	model	patience	lr	StepLR	wd	
3	0.01	0.03436	0.0461	EDSR, f:256, rb: 32	10	1e-4	-	-	simp loss
13 (3B)	0.01	0.03700	0.05071	EDSR, f:128, rb: 64	15	1e-4	0.9	1e-5	
23 (3C)	0.01	0.02544	0.03062	EDSR, f:128, rb: 64	15	1e-4	0.9	1e-5	soft s loss
27 (3D)	0.01	-	-	EDSR, f:128, rb: 64	15	1e-3	0.9	1e-5	soft s loss
28 (3E)	0.01	0.03119	0.04435	EDSR, f:128, rb: 64	15	1e-5	-	1e-5	soft s loss
29 (3F)	0.1	0.02880	0.03863	EDSR, f:128, rb: 64	15	1e-4	0.9	1e-5	soft s loss
30 (3G)	0.001	0.02551	0.03087	EDSR, f:128,	15	1e-4	0.9	1e-5	soft s loss

#experiment	α	lowest validation MAE	lowest validation RMSE	model	patience	lr	StepLR	wd	
				rb: 64					
31 (3H)	0.01	0.02587	0.03248	EDSR, f:128, rb: 64	15	1e-4	0.9	1e-5	soft s loss;
32 (3I)	0.01	0.04171	0.05634	EDSR, f:128, rb: 64	15	1e-4	0.9	1e-5	soft s loss; gradi

- lowest validation MAE: 0.02544, RMSE: 0.03062
- almost same as standard EDSR (MAE: 0.02552, RMSE: 0.03076) (experiments/standard_edsr.md)



Notes

- the soft constraining approach, as average of unshuffled SR gradients, is present in more experiments
- soft constraining (3C) has clearly positive impact on model's performance, compared to standard approach (3, 3B)
- the improvement compared to baseline model is negligible TODO: improve !!
- trying lower/higher learning rates (3D, 3E) does **not** seem to **improve** performance
- trying lower/higher α 's (3F, 3G) does **not** seem to **improve** performance
- trying **higher batch size** (3H) does **not** seem to **improve** performance
- gradient clipping (3I) does **not** seem to **improve** performance
- experiment 3D failed, because there was a NaN value in loss function. We think it was due to high learning rate, which could lead to very large numbers. The soft simple gradient loss, does not use anywhere any division, nor square root, which could lead to some numerical instabilities.
- this is not a soft constraint, as defined by Harder et al. [4]
- the soft constraint approach **COULD BE** a soft constraint, as defined by Beucier et al. [5]



Future work

- ☐ for soft simple gradient loss try MSE
- ☒ for loss try other α s
- ☒ try other soft constraints



Bibliography

1. <https://github.com/sanghyun-son/EDSR-PyTorch/tree/master>

2. KOŠTÁL, Petr; KORDÍK, Pavel; PODSZTAVEK, Ondřej. Downscaling climate projections to 1 km with single-image super resolution. 2025. Available from arXiv: 2509.21399 [cs.CV].
3. Lim et al. (2017)
4. HARDER, Paula; HERNANDEZ-GARCIA, Alex; RAMESH, Venkatesh; YANG, Qidong; SATTIGERI, Prasanna; SZWARCMAN, Daniela; WATSON, Campbell; ROLNICK, David. Hard-Constrained Deep Learning for Climate Downscaling. 2024. Available from arXiv: 2208.05424 [physics.ao-ph].
5. BEUCLER, Tom; PRITCHARD, Michael; RASP, Stephan; OTT, Jordan; BALDI, Pierre; GENTINE, Pierre. Enforcing Analytic Constraints in Neural Networks Emulating Physical Systems. Phys. Rev. Lett. 2021, vol. 126, p. 098302. Available from doi: 10.1103/PhysRevLett.126.09